

PRACTICAL – 2

'''

Implement A star Algorithm for any game search problem.

'''

```
import heapq
```

Heuristic: Manhattan Distance

```
def heuristic(a, b):  
    return abs(a[0] - b[0]) + abs(a[1] - b[1])
```

A* Algorithm

```
def a_star(grid, start, goal):  
    rows, cols = len(grid), len(grid[0])  
  
    open_list = []  
    heapq.heappush(open_list, (0, start))  
  
    came_from = {}  
    g_cost = {start: 0}  
  
    while open_list:  
        _, current = heapq.heappop(open_list)  
  
        if current == goal:  
            path = []  
            while current in came_from:  
                path.append(current)  
                current = came_from[current]  
            path.append(start)  
            path.reverse()  
            return path  
  
        x, y = current  
  
        # 4 possible moves (up, down, left, right)  
        neighbors = [(x+1,y), (x-1,y), (x,y+1), (x,y-1)]  
  
        for nx, ny in neighbors:  
            if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == 0:  
                new_cost = g_cost[current] + 1  
  
                if (nx, ny) not in g_cost or new_cost < g_cost[(nx, ny)]:  
                    g_cost[(nx, ny)] = new_cost  
                    f_cost = new_cost + heuristic((nx, ny), goal)  
                    heapq.heappush(open_list, (f_cost, (nx, ny)))  
                    came_from[(nx, ny)] = current  
  
    return None
```

Input grid size

```
rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))
```

```
print("Enter grid (0 = free, 1 = obstacle):")
grid = []
for _ in range(rows):
    grid.append(list(map(int, input().split())))
```

Start and Goal

```
start = tuple(map(int, input("Enter start position (row col): ").split()))
goal = tuple(map(int, input("Enter goal position (row col): ").split()))
```

Run A*

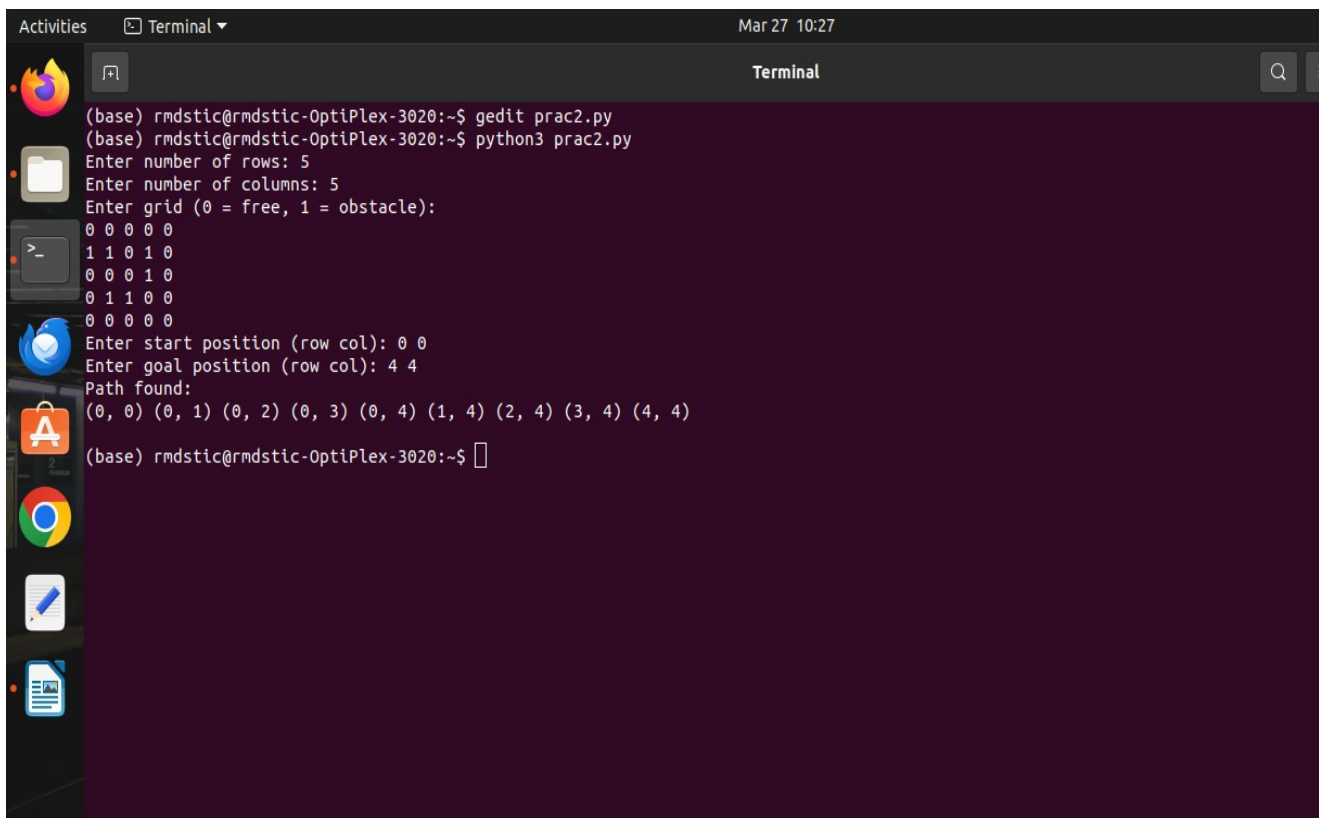
```
path = a_star(grid, start, goal)
```

Output

```
if path:
    print("Path found:")
    for p in path:
        print(p, end=" ")
    print("\n")
```

```
else:
    print("No path exists!")
    print("\n")
```

OUTPUT



The image shows a terminal window titled "Terminal" with a search icon in the top right corner. The window displays the output of a Python script named "prac2.py". The script prompts the user for the number of rows (5) and columns (5), then for a grid where 0 represents free space and 1 represents an obstacle. The grid is as follows:

```
0 0 0 0 0
1 1 0 1 0
0 0 0 1 0
0 1 1 0 0
0 0 0 0 0
```

The script then prompts for a start position (0 0) and a goal position (4 4). It finds a path and displays it as a sequence of coordinates: (0, 0) (0, 1) (0, 2) (0, 3) (0, 4) (1, 4) (2, 4) (3, 4) (4, 4). The terminal window also shows the command prompt and the file manager icon on the left sidebar.

```
(base) rmdstic@rmdstic-OptiPlex-3020:~$ gedit prac2.py
(base) rmdstic@rmdstic-OptiPlex-3020:~$ python3 prac2.py
Enter number of rows: 5
Enter number of columns: 5
Enter grid (0 = free, 1 = obstacle):
0 0 0 0 0
1 1 0 1 0
0 0 0 1 0
0 1 1 0 0
0 0 0 0 0
Enter start position (row col): 0 0
Enter goal position (row col): 4 4
Path found:
(0, 0) (0, 1) (0, 2) (0, 3) (0, 4) (1, 4) (2, 4) (3, 4) (4, 4)
(base) rmdstic@rmdstic-OptiPlex-3020:~$
```